

MUFG Full Stack Analyst Intern

150-Question Comprehensive Test Prep Bank

Every question includes a concise, exam-ready answer or key point. Organized in the same order as the priority checklist from your prep guide. Read each answer out loud once — verbal fluency is what the live/technical rounds actually test.

A. Quantitative Aptitude (7 questions)

- 1. A shopkeeper marks goods 40% above cost price and gives a 25% discount. Find his profit/loss %.**

$SP = 0.75 \times 1.4 \times CP = 1.05 CP \rightarrow 5\% \text{ profit.}$

- 2. Convert 90 km/h to m/s.**

$90 \times 5/18 = 25 \text{ m/s.}$

- 3. A can do a job in 10 days, B in 15 days. How long will they take working together?**

Combined rate = $1/10 + 1/15 = 1/6 \rightarrow 6 \text{ days.}$

- 4. What is the formula for compound interest, and how does it differ from simple interest?**

$CI = P(1+R/100)^T - P$; $SI = PRT/100$. CI compounds on the accumulated amount each period, so $CI > SI$ for $T > 1$.

- 5. Two trains 100 km apart move toward each other at 40 km/h and 60 km/h. When do they meet?**

Relative speed (opposite direction) = 100 km/h $\rightarrow$ time = $100/100 = 1 \text{ hour.}$

- 6. If the price of an item increases by 20% and then decreases by 20%, what is the net % change?**

Net change = $20 - 20 + (20 \times -20)/100 = -4\%$ (net decrease, since successive % changes aren't symmetric).

- 7. What is the difference between permutation and combination?**

Permutation (nPr) counts arrangements where order matters; combination (nCr) counts selections where order doesn't. $nCr = nPr / r!$.

B. Logical Reasoning (7 questions)

- 8. In a syllogism 'All A are B, Some B are C' — can you conclude 'Some A are C'?**

No. This is a classic invalid-conclusion trap — draw the Venn diagram; the overlap between B and C need not include any A.

- 9. What's the best strategy for seating-arrangement/puzzle questions?**

Read all clues fully first, place the most absolute/restrictive clue (e.g., 'X is at one end') before relative clues, and draw the diagram as you go.

- 10. Explain the difference between 'coding-decoding' and 'blood relation' question types.**

Coding-decoding maps letters/numbers/symbols to a pattern (e.g., shift ciphers); blood relations require building a family tree from a chain of stated relationships.

- 11. What is a 'data sufficiency' question and how should you approach it?**

It asks whether given statement(s) are enough to answer a question — without actually solving it. Check each statement independently first, then together, rather than jumping to solve.

- 12. A man walks 5 km north, then 3 km east, then 5 km south. How far is he from the start?**

The north and south legs cancel out; he ends up 3 km east of start $\rightarrow$ distance = 3 km.

- 13. What is the trap in most 'statement and assumption' questions?**

Confusing a reasonable inference with an unstated, non-obvious assumption the argument actually depends on — a valid assumption must be necessary for the conclusion to hold.

- 14. How do you solve a number series like 2, 6, 12, 20, 30, ...?**

Look at differences: 4, 6, 8, 10 — second differences are constant (2), meaning it's a quadratic pattern ($n \times (n+1)$); next term = 42.

C. Verbal Ability (5 questions)

- 15. What's the fastest way to tackle a reading-comprehension passage under time pressure?**

Read the questions first, then skim the passage specifically hunting for those answers rather than reading it fully top to bottom.

- 16. How do you solve a para-jumble question efficiently?**

Identify the sentence that reads like a general opening statement and the one that reads like a conclusion first, then fit the middle sentences by locating pronoun/logical links.

- 17. What's a common subject-verb agreement trap in sentence-correction questions?**

A long phrase between subject and verb (e.g., 'The list of items is...') tricking you into matching the verb to the nearer noun instead of the true subject.

18. Give an example of commonly confused synonym/antonym pairs tested in these exams.

e.g., 'Candid' (honest/frank) vs 'Concealed'; 'Diligent' (hardworking) vs 'Indolent' (lazy) — expect business/formal-register vocabulary.

19. How should you handle an unfamiliar word in a fill-in-the-blank question?

Use context clues from the rest of the sentence to infer tone/meaning (positive vs negative, formal vs casual) even without knowing the exact word.

D. Core Java & OOP (18 questions)

20. What are the four pillars of OOP? Give a one-line example of each.

Encapsulation (private fields + getters/setters), Abstraction (interface hiding implementation), Inheritance (subclass extends superclass), Polymorphism (same method behaves differently per object type, e.g., method overriding).

21. Difference between method overloading and overriding?

Overloading: same name, different parameter list, resolved at compile time (same class). Overriding: subclass redefines a superclass method with the same signature, resolved at runtime.

22. Difference between abstract class and interface?

Abstract class can have constructors, state, and partial implementation; interface (Java 8+) can have default/static methods but no instance state — use interface for a 'can-do' contract, abstract class for shared 'is-a' code.

23. Explain final, finally, and finalize().

final: makes a variable constant / method non-overridable / class non-extendable. finally: block that always executes after try/catch. finalize(): deprecated method called by GC before object removal.

24. Difference between == and .equals() in Java?

== compares references (memory addresses) for objects, or primitive values directly. .equals() compares logical/value equality, and can be overridden (e.g., String, custom classes).

25. Why must you override hashCode() when you override equals()?

Hash-based collections (HashMap/HashSet) rely on the contract that equal objects must produce equal hash codes; violating this breaks lookups and causes duplicate 'equal' entries.

26. Explain Java's exception hierarchy.

Throwable → Error (serious, unrecoverable, e.g., OutOfMemoryError) and Exception → checked (must be declared/caught, e.g., IOException) vs unchecked/Runtime Exception (e.g., NullPointerException).

27. ArrayList vs LinkedList — when would you choose each?

ArrayList: backed by a dynamic array, O(1) random access, good for frequent reads. LinkedList: doubly-linked list, O(1) insert/delete at ends, better for frequent insertions/deletions in the middle... though ArrayList is usually faster in practice for most workloads.

28. Explain how HashMap handles hash collisions internally.

Each bucket holds entries with the same hash (as a linked list); Java 8+ converts a bucket's list to a balanced tree once it exceeds 8 entries, improving worst-case lookup from O(n) to O(log n).

29. What is the default load factor of a HashMap and what does it control?

0.75 — once the map is 75% full, it resizes (doubles capacity) and rehashes all entries to keep lookups fast.

30. Difference between Thread and Runnable in Java?

Implementing Runnable and passing it to a Thread is preferred over extending Thread directly, since Java doesn't support multiple inheritance and Runnable keeps the class free to extend something else.

31. What is the synchronized keyword used for?

It ensures only one thread can execute a block/method on a given object at a time, preventing race conditions on shared mutable state.

32. What does the volatile keyword do?

Ensures a variable's value is always read from/written to main memory (not a thread-local CPU cache), so changes are visible across threads immediately — but it does NOT provide atomicity for compound operations.

33. What is a deadlock and how can it be avoided?

Two or more threads waiting on each other's locks indefinitely. Avoid by acquiring locks in a consistent global order, using timeouts, or minimizing nested locking.

34. What is type erasure in Java generics?

Generic type information is removed at compile time and replaced with Object (or bounds) at runtime — generics exist mainly for compile-time type safety.

35. Name two Java 8 features and briefly explain each.

Streams API (functional-style operations like map/filter/reduce on collections) and Lambda expressions (concise anonymous functions implementing functional interfaces like Runnable or Comparator).

36. Why is the String class immutable in Java?

For security, thread-safety, and to allow the JVM's String pool to safely cache/reuse String literals — mutable strings would break hashcode caching and pool sharing.

37. StringBuilder vs StringBuffer — what's the key difference?

StringBuffer is synchronized (thread-safe but slower); StringBuilder is not synchronized (faster, use when thread-safety isn't needed).

E. J2EE, Spring & REST APIs (7 questions)

38. Describe the Servlet lifecycle.

init() — called once when the servlet is loaded, service() — handles each incoming request (dispatches to doGet/doPost), destroy() — called once before the servlet is unloaded.

39. What is dependency injection and why does Spring use it?

A pattern where an object's dependencies are supplied externally by the Spring IoC container rather than the object creating them itself, improving testability and reducing tight coupling.

40. What does @RestController do differently from @Controller in Spring?

@RestController combines @Controller + @ResponseBody, meaning return values are serialized directly into the HTTP response body (e.g., as JSON) instead of being resolved to a view template.

41. What makes an API RESTful? Name the core principles.

Statelessness (no client session stored server-side between requests), resource-based URLs, correct use of HTTP verbs, and representations (usually JSON) of resource state.

42. Explain the difference between PUT and PATCH.

PUT replaces the entire resource with the payload given; PATCH applies a partial update, modifying only the specified fields.

43. Why use PreparedStatement instead of Statement in JDBC?

PreparedStatement pre-compiles SQL with placeholder parameters, treating user input strictly as data rather than executable SQL — this prevents SQL injection, unlike string-concatenated Statement queries.

44. What HTTP status code would you return for a successful POST that creates a resource, and for a validation failure?

201 Created for a successful resource creation; 400 Bad Request for a validation failure (vs 401 Unauthorized / 403 Forbidden for auth issues, 404 for missing resources, 500 for server errors).

F. JavaScript Fundamentals (18 questions)

45. Difference between var, let, and const?

var is function-scoped and hoisted as undefined; let/const are block-scoped and sit in a 'temporal dead zone' until their declaration line runs; const additionally can't be reassigned.

46. What does this print? for (var i=0;i<3;i++){ setTimeout(()=>console.log(i)) }

3, 3, 3 — because var is function-scoped, all three callbacks share the same 'i', which is 3 by the time they run. Using let instead prints 0, 1, 2 since each iteration gets its own block-scoped binding.

47. What is hoisting?

JS moves variable/function declarations to the top of their scope during compilation — function declarations are fully hoisted (usable before their line), var is hoisted as undefined, let/const are hoisted but stay uninitialized.

48. Difference between map, filter, and reduce?

map transforms each element into a new array of the same length; filter returns a new array containing only elements passing a test; reduce accumulates all elements into a single value.

49. What is event delegation and why is it useful?

Attaching a single event listener to a parent element instead of each child, using event bubbling to detect which child triggered it — more memory-efficient and works automatically for dynamically-added children.

50. Explain the difference between event bubbling and event capturing.

Bubbling: the event fires on the target first, then propagates upward through ancestors. Capturing: the event fires on ancestors first, moving downward to the target. `addEventListener`'s third argument controls which phase you listen on.

51. What is prototypal inheritance?

Every JS object has an internal `[[Prototype]]` link to another object; property lookups walk up this prototype chain if not found on the object itself. ES6 'class' syntax is sugar over this mechanism, not true classical inheritance.

52. Difference between == and === in JavaScript?

`==` compares with type coercion (converts operands to a common type first); `===` compares strictly, with no type conversion — always prefer `===` to avoid coercion bugs.

53. How does the 'this' keyword behave differently in a regular function vs an arrow function?

In a regular function, 'this' depends on how the function is called (call-site); in an arrow function, 'this' is lexically inherited from the enclosing scope at definition time and never changes.

54. What is a closure? Give a practical example.

A function that retains access to its outer (enclosing) function's variables even after the outer function has returned. Example: `function makeCounter(){ let c=0; return ()=>{c++;} — 'c' stays private across calls.`

55. Explain the JavaScript event loop briefly.

JS runs on a single thread with a call stack; async callbacks queue up — microtasks (Promises) are fully drained before the next macrotask (`setTimeout`, UI events) on each loop tick, which is why `Promise.then()` runs before a `0ms setTimeout`.

56. Difference between Promise.all and Promise.race?

`Promise.all` resolves when ALL given promises resolve (or rejects as soon as any one rejects); `Promise.race` resolves/rejects as soon as the FIRST promise settles, regardless of outcome.

57. What does async/await do under the hood?

Syntactic sugar over Promises — 'await' pauses execution of the async function (without blocking the thread) until the awaited Promise settles, making asynchronous code read like synchronous code.

58. Explain destructuring with an example.

Extracts values from arrays/objects into variables concisely: `const {name, age} = user;` or `const [first, second] = arr;`

59. What's the difference between the spread and rest operators (both use ...)?

Spread expands an iterable into individual elements (e.g., in a function call or array literal); rest collects multiple arguments/remaining elements into a single array (e.g., `function(...args)`).

60. Write a simple debounce function.

`function debounce(fn, delay){ let t; return (...a)=>{ clearTimeout(t); t=setTimeout(()=>fn(...a), delay); }; } — delays execution until 'delay' ms have passed since the last call.`

61. What's the difference between null and undefined?

undefined means a variable has been declared but not assigned a value (or a property doesn't exist); null is an explicit, intentional 'no value' assignment made by the developer.

62. What are JS modules (import/export) and why use them?

They let you split code into reusable, independently-scoped files, explicitly exporting what should be public and importing only what's needed — avoiding global namespace pollution.

G. React.js (Hooks, Redux/Flux) (15 questions)

63. What is the Virtual DOM and why does React use it?

An in-memory lightweight representation of the real DOM; React diffs the new virtual tree against the previous one and applies only the minimal set of real DOM changes, which is faster than re-rendering the whole DOM directly.

64. Why are 'key' props important in a list, and what breaks without them?

Keys let React match array items across renders to correctly reuse/update DOM nodes. Without stable keys (or using array index on a reorderable list), state can get mismatched onto the wrong item after reordering/insertion.

65. Explain the useEffect dependency array's three behaviors.

Empty array [] → effect runs once after initial mount only. No array at all → effect runs after every render. [dep1, dep2] → effect re-runs only when one of those dependencies changes between renders.

66. Difference between useMemo and useCallback?

useMemo memoizes a computed VALUE (recomputed only when dependencies change); useCallback memoizes a FUNCTION reference itself, useful for passing stable callbacks to memoized child components.

67. What is the difference between controlled and uncontrolled components?

Controlled: form input value is driven by React state (via value + onChange). Uncontrolled: the DOM itself holds the input's value, accessed via a ref when needed.

68. Explain the unidirectional data flow in Redux.

Action (describes what happened) → dispatched to Reducer (a pure function computing new state from old state + action) → updates the central Store → View re-renders based on new store state.

69. Why must Redux reducers be pure functions?

Purity (same input always → same output, no side effects) makes state changes predictable, testable, and enables features like time-travel debugging by replaying actions deterministically.

70. When would you choose Context API over Redux?

Context suits simple, low-frequency global state (e.g., theme, logged-in user) without middleware needs; Redux scales better for complex, frequently-updated state with devtools, middleware (like async thunks), and larger apps.

71. What does React.memo do?

Wraps a functional component to skip re-rendering if its props haven't changed (shallow comparison), used for performance optimization on expensive components.

72. How does React's reconciliation/diffing algorithm decide what to update?

It compares elements by type first (different types → full subtree replace); for same-type elements it compares/updates changed props/attributes, and for lists it uses keys to match items across renders.

73. What is 'lifting state up' in React?

Moving shared state from child components up to their closest common ancestor, so multiple children can stay in sync by reading/updating that shared state via props.

74. What is JSX and how does it actually work under the hood?

JSX is syntactic sugar that transpiles (via Babel) into React.createElement(type, props, children) calls — it's not HTML, it's just a more readable way to write nested function calls that build the element tree.

75. What are custom hooks and why would you write one?

A custom hook is a JS function (name starting with 'use') that composes other hooks to extract and reuse stateful logic across multiple components without duplicating code.

76. How would you fetch data in a functional component and handle loading/error states?

Use useState for data/loading/error, and useEffect (with an empty dependency array for mount-time fetch) to trigger the async call, updating state in .then().catch() or via try/catch with async/await, remembering to guard against state updates after unmount.

77. What is code-splitting in React and how is it done?

Breaking your bundle into smaller chunks loaded on demand instead of all upfront, typically via React.lazy() + Suspense combined with dynamic import(), improving initial load performance.

H. HTML & CSS (7 questions)

78. Explain the CSS box model.

Every element is a box made of content, padding, border, and margin (outward in that order). content-box (default) sizes width/height to content only; border-box includes padding+border within the declared width/height.

79. Flexbox vs Grid — when would you use each?

Flexbox is best for one-dimensional layouts (a single row or column, e.g., a navbar); Grid is best for two-dimensional layouts (rows AND columns simultaneously, e.g., a full page layout).

80. Explain the difference between position: relative, absolute, fixed, and sticky.

relative: offset from its own normal position. absolute: positioned relative to the nearest positioned ancestor. fixed: positioned relative to the viewport, stays put on scroll. sticky: behaves relative until a scroll threshold, then sticks like fixed.

81. How is CSS specificity calculated?

Roughly in order of increasing weight: element/tag selectors < class/attribute/pseudo-class selectors < ID selectors < inline styles, with !important overriding everything (best avoided).

82. Why is semantic HTML important?

It improves accessibility (screen readers understand structure), SEO (search engines weigh semantic tags), and code readability, compared to using generic s for everything.

83. What's the difference between rem, em, and px units?

px is a fixed absolute unit; em is relative to the parent element's font-size (compounds when nested); rem is relative to the root (html) font-size only, making it more predictable for consistent scaling.

84. What does 'mobile-first' responsive design mean?

Writing your base CSS for the smallest screen first, then using min-width media queries to progressively add complexity for larger screens, rather than the reverse.

I. SQL & Databases (12 questions)

85. Explain INNER JOIN vs LEFT JOIN vs FULL OUTER JOIN.

INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from the left table plus matches from the right (NULLs where no match). FULL OUTER JOIN returns all rows from both tables, matched where possible.

86. Difference between WHERE and HAVING?

WHERE filters individual rows BEFORE grouping/aggregation; HAVING filters groups AFTER a GROUP BY aggregation (e.g., HAVING COUNT(*) > 5).

87. Write a query to find the second-highest salary in an Employee table.

SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee); — or use DENSE_RANK() OVER (ORDER BY salary DESC) for the general Nth-highest case.

88. What is database normalization and why is it used?

Organizing tables to reduce data redundancy and avoid update/insert/delete anomalies, typically up to 3NF, by ensuring each non-key attribute depends only on the whole primary key.

89. What is an index and what's the tradeoff of adding one?

A separate data structure (often a B-tree) that speeds up read/lookup queries on a column, at the cost of slower writes (INSERT/UPDATE/DELETE must also update the index) and extra storage.

90. Difference between a clustered and non-clustered index?

A clustered index physically orders the table's data rows by the indexed column (only one per table); a non-clustered index is a separate structure with pointers back to the actual rows (a table can have many).

91. What is the difference between a primary key and a foreign key?

A primary key uniquely identifies each row in its own table (unique, non-null); a foreign key is a column referencing another table's primary key, enforcing referential integrity between tables.

92. Explain the ACID properties of a database transaction.

Atomicity (all-or-nothing), Consistency (valid state before and after), Isolation (concurrent transactions don't interfere), Durability (committed changes survive a crash).

93. What is a dirty read, and which isolation level prevents it?

Reading uncommitted data from another in-progress transaction that might later be rolled back. Read Committed (and stricter levels) prevents dirty reads by only exposing committed data.

94. How does a PreparedStatement prevent SQL injection?

It separates the SQL structure from the data — parameters are bound as literal values rather than concatenated into the query string, so malicious input can never alter the query's logic.

95. What is a subquery, and give an example use case.

A query nested inside another query, e.g., finding employees earning more than the department average: `SELECT * FROM Employee WHERE salary > (SELECT AVG(salary) FROM Employee)`.

96. What's the difference between DELETE, TRUNCATE, and DROP?

DELETE removes specific rows (can be rolled back, fires triggers, keeps table structure); TRUNCATE removes all rows fast (usually can't be rolled back, resets identity); DROP removes the entire table/object structure permanently.

J. Data Structures & Algorithms (12 questions)

97. What's the time complexity of accessing an element in an array vs a linked list?

Array: $O(1)$ — direct index access. Linked list: $O(n)$ — must traverse from the head to reach the target node.

98. Explain the difference between a stack and a queue.

Stack: LIFO (Last-In-First-Out), operations push/pop from one end. Queue: FIFO (First-In-First-Out), enqueue at the back, dequeue from the front.

99. How do you check if a string is a palindrome?

Compare characters from both ends moving inward (two-pointer approach), or compare the string to its reverse — $O(n)$ time either way.

100. Explain the two-sum problem approach.

Use a hash map to store each visited number and its index; for each new number, check if (target – number) already exists in the map — gives $O(n)$ time instead of $O(n^2)$ brute force.

101. What is Big-O notation and why does it matter?

A way to describe how an algorithm's runtime/space grows relative to input size in the worst case, letting you compare algorithm efficiency independent of hardware.

102. Explain the difference between Merge Sort and Quick Sort.

Merge Sort: divide-and-conquer, always $O(n \log n)$, stable, needs $O(n)$ extra space. Quick Sort: divide-and-conquer via pivot partitioning, average $O(n \log n)$ but worst case $O(n^2)$, typically in-place and faster in practice due to cache locality.

103. How do you detect a cycle in a linked list?

Floyd's cycle detection (slow/fast pointer, aka tortoise and hare): move one pointer one step and another two steps; if they ever meet, there's a cycle.

104. What's the difference between BFS and DFS on a graph?

BFS explores level by level using a queue (good for shortest path in unweighted graphs); DFS explores as deep as possible before backtracking, usually using a stack or recursion.

105. Explain binary search and its time complexity.

Repeatedly halves a SORTED array's search space by comparing the target to the middle element — $O(\log n)$ time, since the input array size must be sorted beforehand.

106. What is recursion, and what are its two essential components?

A function calling itself to solve a smaller instance of the same problem — needs a base case (stopping condition) and a recursive case (progress toward the base case) to avoid infinite recursion/stack overflow.

107. How would you find the missing number in an array containing 1 to N with one number missing?

Compute expected sum $N(N+1)/2$, subtract the actual sum of the array — the difference is the missing number. $O(n)$ time, $O(1)$ space.

108. What is a hash map's average time complexity for insert/lookup/delete, and when does it degrade?

Average $O(1)$ for all three, assuming a good hash function distributes keys evenly. It degrades toward $O(n)$ in the worst case with many collisions (e.g., a poor hash function or adversarial input) before resizing/treeification kicks in.

K. System Design / Distributed Systems (6 questions)

109. Describe a standard 3-tier architecture.

Presentation tier (client/UI) → Application tier (business logic/API server) → Data tier (database) — each layer can scale and be modified independently.

110. Monolith vs microservices — what's the core tradeoff?

Monolith: simpler to build/deploy/debug initially, but scaling and deploying individual features independently is hard. Microservices: independent scaling/deployment per service, but adds network latency, distributed-system complexity, and operational overhead.

111. Why would you use a message queue like Kafka between services?

It decouples producers and consumers, enabling asynchronous, event-driven communication that buffers against traffic spikes and lets each side fail/scale independently.

112. What is a load balancer and why is it needed?

A component that distributes incoming requests across multiple server instances (e.g., round robin, least connections) to improve availability, throughput, and fault tolerance.

113. Why does statelessness matter for a REST API's scalability?

Since no session state is stored on a specific server, any request can be routed to any server instance, making horizontal scaling (adding more servers) straightforward without sticky-session complexity.

114. What's the fundamental difference between a container (Docker) and a virtual machine?

Containers share the host OS kernel and only virtualize the application layer (lightweight, fast startup); VMs virtualize the entire hardware stack including a full guest OS (heavier, but stronger isolation).

L. Git (5 questions)

115. Difference between git merge and git rebase?

merge creates a new merge commit combining both histories (preserves full history, non-linear); rebase replays your commits on top of another branch's tip, producing a linear (but rewritten) history — never rebase a shared/public branch.

116. How do you resolve a merge conflict?

Git marks conflicting sections in the file with <<<<<<, =====, >>>>>> markers; manually edit to the correct final content, remove the markers, then git add the file and commit (or continue the rebase/merge).

117. Difference between git fetch and git pull?

git fetch downloads remote changes without merging them into your current branch; git pull is fetch + merge (or rebase) combined into a single step.

118. What is a detached HEAD state?

When HEAD points directly to a specific commit instead of a branch reference — commits made here aren't attached to any branch and can be lost unless you create a new branch from that point.

119. What does git stash do and when would you use it?

Temporarily shelves uncommitted changes so you can switch branches cleanly, then git stash pop/apply restores them later — useful when you need to quickly context-switch without committing half-finished work.

M. Application Security (OWASP) (7 questions)

120. What is SQL Injection and how do you prevent it?

Injecting malicious SQL through unsanitized user input to manipulate a query's logic. Prevent it with parameterized queries/PreparedStatements, never string-concatenating raw input into SQL.

121. What is XSS (Cross-Site Scripting)?

Injecting malicious script into a page that then executes in another user's browser session. Prevent with output encoding, avoiding raw HTML injection (e.g., React's dangerouslySetInnerHTML), and Content-Security-Policy headers.

122. What is CSRF and how is it mitigated?

An attack tricking a logged-in user's browser into making an unwanted request to another site using their existing session. Mitigated with anti-CSRF tokens and SameSite cookie attributes.

123. What is CORS and why does the browser enforce it?

Cross-Origin Resource Sharing — a browser policy restricting which origins (domains) can call your API from client-side JS, preventing a malicious site from silently using a victim's authenticated session against another site.

124. What is a JWT and what are its three parts?

JSON Web Token — a compact, self-contained token for stateless auth, made of header.payload.signature (base64-encoded), where the signature verifies the token hasn't been tampered with.

125. Why is HTTPS/TLS important for a web application?

It encrypts data in transit between client and server, preventing eavesdropping/tampering (man-in-the-middle attacks) on sensitive data like credentials or session tokens.

126. What is the principle of least privilege?

Every user, service, or process should be granted only the minimum access/permissions necessary to perform its function — limiting the blast radius if that account or service is compromised.

N. Agile / Scrum / Waterfall (4 questions)

127. What are the main Scrum ceremonies and their purpose?

Sprint Planning (decide what to build this sprint), Daily Standup (quick sync on progress/blockers), Sprint Review (demo completed work to stakeholders), Sprint Retrospective (reflect on process improvements).

128. When would Waterfall be preferable to Agile?

When requirements are well-understood and unlikely to change, and the project needs strict upfront documentation/compliance (e.g., certain regulated or fixed-scope contract projects).

129. Difference between Kanban and Scrum?

Scrum uses fixed-length time-boxed sprints with defined roles/ceremonies; Kanban is a continuous flow model with WIP (work-in-progress) limits and no mandatory fixed iteration cadence.

130. What is a Product Backlog?

A prioritized, evolving list of everything that might be needed in the product (features, fixes, tech debt), owned and ordered by the Product Owner, from which sprint work is pulled.

O. AWS, Docker & CI/CD (6 questions)

131. What's the difference between EC2 and Lambda?

EC2 is a persistent virtual server you manage and pay for continuously; Lambda is serverless — code runs only in response to triggered events, and you pay only for actual execution time.

132. What is S3 used for, and how does it differ from EBS?

S3 is object storage (files/blobs accessed via API, highly durable, not attached to a specific server); EBS is block storage attached directly to a single EC2 instance, like a virtual hard disk.

133. What is IAM in AWS?

Identity and Access Management — controls who (users, roles, services) can access which AWS resources and what actions they can perform, ideally following least-privilege.

134. What problem does Docker solve?

It packages an application with all its dependencies into a portable, consistent container image, eliminating 'it works on my machine' issues across dev, test, and production environments.

135. What is Infrastructure as Code (e.g., Terraform)?

Defining cloud infrastructure declaratively in version-controlled config files instead of manual console setup, making environments reproducible, auditable, and consistent across teams.

136. What's the difference between Continuous Integration and Continuous Deployment?

CI automatically builds and tests code on every commit to catch integration issues early; Continuous Deployment automatically ships changes that pass those tests straight to production (Continuous Delivery is the same but with a manual approval gate before release).

P. LLMs / GenAI Basics (5 questions)

137. What is the difference between prompting and fine-tuning an LLM?

Prompting shapes model output at inference time via instructions/examples without changing model weights; fine-tuning further trains the model's weights on a specific dataset — more durable and specialized, but costlier.

138. What is RAG (Retrieval-Augmented Generation) and why use it?

Retrieving relevant external data at query time and feeding it into the model's context, grounding responses in accurate/up-to-date/private information and reducing hallucination.

139. What makes something an 'AI agent' rather than a simple chatbot call?

An agent can reason toward a goal across multiple steps, decide to call external tools/APIs, observe results, and iterate autonomously — not just respond once to a single prompt.

140. How would you get reliable structured (e.g., JSON) output from an LLM?

Explicitly instruct the model to output ONLY valid JSON matching a given schema/example, and validate/parse the response with error handling on the backend in case of malformed output.

141. Name a few major LLM providers/model families.

GPT (OpenAI), Claude (Anthropic), Gemini (Google), and open-source options like Llama and Mistral.

Q. HR & Behavioral (9 questions)

142. Tell me about yourself.

Structure: current status (student/role) → most relevant technical project/experience → what you're looking for and why it connects to this role. Keep it under 90 seconds, end by transitioning naturally into 'why MUFG'.

143. What are your strengths and weaknesses?

Pick a strength directly relevant to this role (e.g., persistence in debugging complex issues) with a quick proof example; for weakness, pick something real but show the concrete step you're taking to improve it — avoid fake weaknesses like 'I work too hard'.

144. Why MUFG? Why banking/financial services?

Connect genuinely: MUFG is Japan's largest bank with a global technology footprint — mention your interest in large-scale, high-reliability systems (fintech/trading tech) and how that maps to what a bank's tech org builds. Keep it specific and short, not a generic script.

145. Tell me about a challenge you faced in a project and how you handled it.

Use STAR: briefly set the Situation/Task, focus most of the answer on the specific Action you took (your reasoning, not just what happened), and end with a concrete Result/learning.

146. Describe a time you disagreed with a teammate or had a technical disagreement.

Show you listened to their view, explained your reasoning with evidence/data rather than just opinion, and reached a resolution collaboratively — avoid framing it as 'I was right and they were wrong'.

147. How do you handle tight deadlines or ambiguous requirements?

Give a concrete example of prioritizing ruthlessly (what's truly essential vs nice-to-have), communicating proactively about tradeoffs, and adjusting scope rather than silently missing a deadline.

148. What do you expect from management?

Clear priorities and context on the 'why' behind tasks, timely feedback, and enough autonomy to solve problems your own way once expectations are set — keep it collaborative, not demanding.

149. Describe a time you had to learn something new quickly.

Pick a real technical example (a new framework, API, or concept for a project), briefly explain your learning approach (docs → small experiment → applying it), and the outcome it enabled.

150. Do you have any questions for us? (always prepare 1-2)

Ask something genuine and specific — e.g., about the team's tech stack, how interns are mentored, or what a successful first 90 days looks like in this role. Avoid asking anything answerable from the website alone.